

Java Programing: An Introduction to OOP



Faculty of Computer Science and Mathematics, University of Kufa, Najaf, IRAQ.

1.6 Java Arrays المصفوفات في لغة الجافا

المصفوفة :

An array is a container that holds data (values) of one single type. For example, you can create an array that can hold 100 values of `int` type.

نقوم بتعريف المصفوفة بالشكل التالي:

Here's how you can declare an array in Java:

```
dataType[] arrayName;
```

Let's take the above example again.

```
double[] data; ex: data = new Double[10];
```

example:

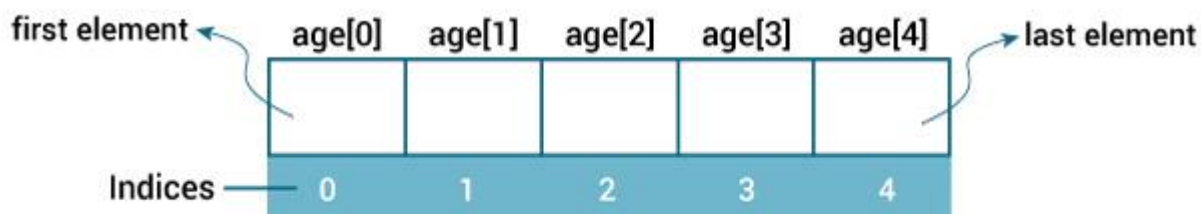
Let's take another example:

```
int[] age;
age = new int[5];
```

Java Array Index

You can access elements of an array by using indices الفهرس. Let's consider previous example.

```
int[] age = new int[5];
```



Array age of length 5

The first element of array is `age[0]`, second is `age[1]` and so on.

If the length of an array is `n`, the last element will be `arrayName[n-1]`. Since the length of `age` array is 5, the last element of the array is `age[4]` in the above example.

Note: The default initial value of elements of an array is `0` for numeric types and `false` for `boolean`. We can demonstrate this:

Example: استخدم المثال التالي لقراءة 5 اعداد وطباعتها اما استخدام صناديق الحوار او من الكيبورد

```
class ArrayExample {
    public static void main(String[] args) {

        int[] age = new int[5];

        System.out.println(age[0]);
        System.out.println(age[1]);
        System.out.println(age[2]);
        System.out.println(age[3]);
        System.out.println(age[4]);
    }
}
```

```
for (int i = 0; i < 5; ++i) {
    System.out.println(age[i]);
}
```

How to access array elements?

كيف نصل الى اي عنصر داخل المصفوفة

You can easily access and alter array elements by using its numeric index. Let's take an example.

```
class ArrayExample {
    public static void main(String[] args) {

        int[] age = new int[5];

        // insert 14 to third element اضافة القيمة 14 الى الخلية الثانية في المصفوفة
        age[2] = 14;

        // insert 34 to first element اضافة القيمة 34 الى الخلية الاولى في المصفوفة
        age[0] = 34;

        for (int i = 0; i < 5; ++i) {
            System.out.println("Element at index " + i + ": " + age[i]);
        }
    }
}
```

Output

```
Element at index 0: 34
Element at index 1: 0
Element at index 2: 14
Element at index 3: 0
Element at index 4: 0
```

ملاحظات مهمة:

- ١ - يجب أن يكون رقم الموقع (Index) عند التعامل مع المصفوفة عدداً صحيحاً موجباً.
- ٢ - يجب أن لا نتجاوز عدد المواقع المحجوزة للمصفوفة عند استخدامها.
- ٣ - إذا لم نحدد قيمة ابتدائية للمصفوفة فيجب أن نستخدم الكلمة المحجوزة (new) لحجز مواقع للمصفوفة كما ذكر سابقاً.
- ٤ - إذا لم تعطى المصفوفة قيمة ابتدائية عند تعريفها فإنها تأخذ القيم التلقائية (Default Value) كقيم ابتدائية وذلك حسب النوع (Type) الذي حدد للمصفوفة. كما ذكر سابقاً.
- ٥ - نستطيع معرفة عدد المواقع المحجوزة للمصفوفة من خلال كتابة اسم المصفوفة ثم نقطة ثم length. (مثال: array1.length، من خلال هذه الجملة نستطيع معرفة عدد المواقع المحجوزة للمصفوفة array1).
- ٦ - نستطيع استخدام أحد الأشكال التالية لتعريف وحجز مواقع المصفوفة:

```
1. int array[]=new int[5];
```

```
2. int array[];
   array=new int[5];
```

```
3. int [] array = new int[5];
```

```
4. int [] array;
   array=new int[5];
```

- ٧ - نستطيع استخدام أحد الأشكال التالية لتخزين القيم الابتدائية في المصفوفة:

```
1. int array[]={5, 3, 8, 9, 2};
```

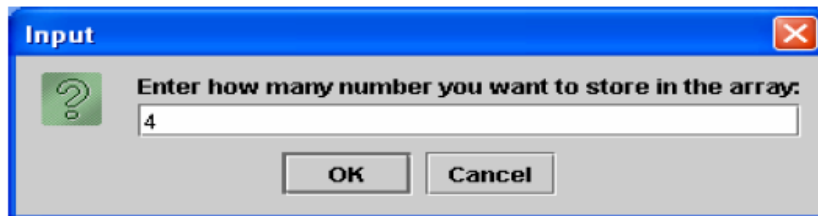
```
2. int array[]=new int[] {5, 3, 8, 9, 2};
```

مثال حول المصفوفة ذات البعد الواحد .. يقوم بادخال حجم المصفوفة من صندوق الحوار وكذلك مليء معلومات المصفوفة من صندوق الحوار .. وبعدها يقوم بطباعة المصفوفة على صندوق الحوار وايجاد الاعداد الفردية في المصفوفة ان وجدت.

```
package example;
import javax.swing.JOptionPane;
import java.util.Scanner;
import javax.swing.*;
public class general
{
    public static void main(String[] args)
    {
        String s, output, title, str1, str2;
        str1="Enter how many number you want to store in the array:";
        str2="Enter number # ";
        int n, odd=0;
        s=JOptionPane.showInputDialog(str1);
        n=Integer.parseInt(s);
        int [] arr=new int[n];
        output= " ";
        for(int i=0; i<arr.length; i++){
            s=JOptionPane.showInputDialog(str2+(i+1)+":");
            arr[i]=Integer.parseInt(s);
            output+=arr[i]+"\\n ";
        } //end for
        for(int i=0; i<arr.length; i++)
            if(arr[i]%2==1) odd++; // end for
        title="The results are)";
        output+="\\nThere are "+odd+" odd numbers in the array";
        JOptionPane.showMessageDialog(null, output, title,JOptionPane.INFORMATION_MESSAGE);
        System.exit(0);
    } //end main
} //end class array3
```

Activate Wind
Go to Settings to a

Output

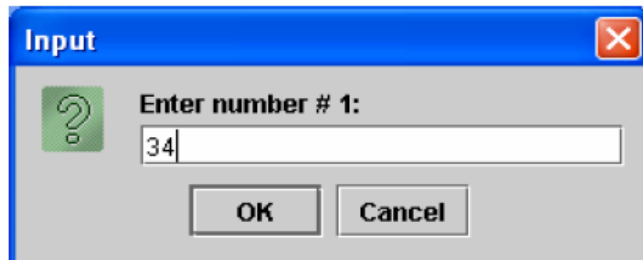


Input

Enter how many number you want to store in the array:

4

OK Cancel

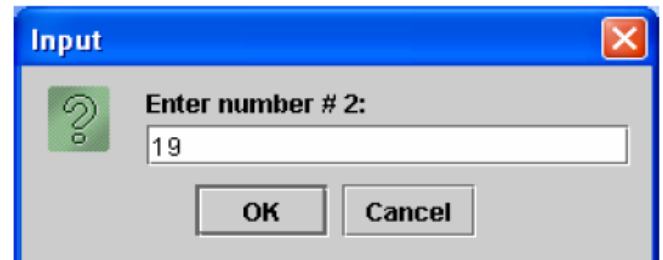


Input

Enter number # 1:

34

OK Cancel

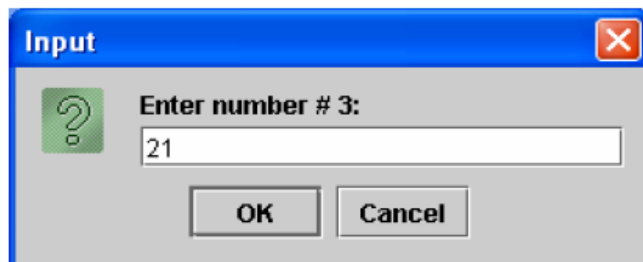


Input

Enter number # 2:

19

OK Cancel

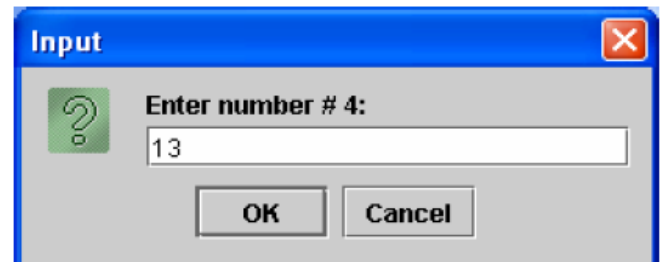


Input

Enter number # 3:

21

OK Cancel

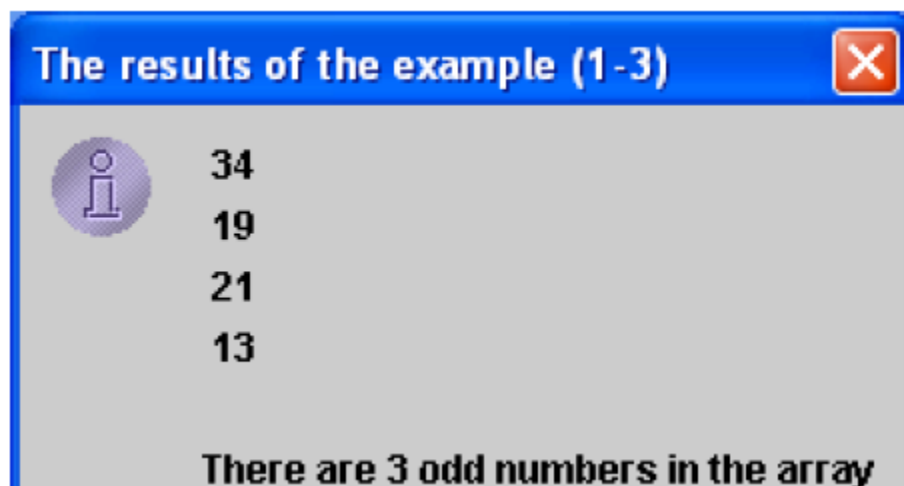


Input

Enter number # 4:

13

OK Cancel



The results of the example (1-3)

34
19
21
13

There are 3 odd numbers in the array

المصفوفات ذات البعدين (Two-Dimensional Arrays):

في لغة جافا يمكن تعريف مصفوفات ذات أكثر من بعد واحد، وكمثال على ذلك: تعريف المصفوفات ذات البعدين. ونستطيع القول بأن المصفوفة ذات البعدين هي عبارة عن جدول يحتوي على صفوف وأعمدة، انظر الشكل والمثال التالي يوضح كيفية تعريف مصفوفة ذات بعدين وحجز مواقع لها:

```
1. int b[][];
2. b = new int[ 3 ][ 4 ];
```

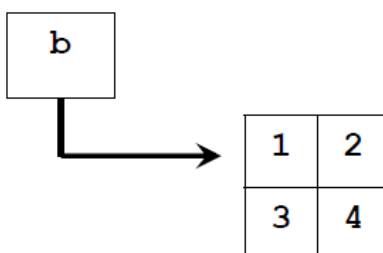
في السطر الأول تم تعريف مصفوفة ذات بعدين، وفي الصف الثاني تم حجز مواقع لهذه المصفوفة بحيث تحتوي على ثلاث صفوف كل صف منها يحتوي على ثلاثة أعمدة. والشكل يوضح المصفوفة b وأرقام مواقعها.

	العمود 0	العمود 1	العمود 2	العمود 3
الصف 0	b[0][0]	b[0][1]	b[0][2]	b[0][3]
الصف 1	b[1][0]	b[1][1]	b[1][2]	b[1][3]
الصف 2	b[2][0]	b[2][1]	b[2][2]	b[2][3]

والمثال التالي يوضح كيفية تعريف مصفوفة ذات بعدين وإعطائها قيماً ابتدائية:

```
int b[][] = { { 1, 2 }, { 3, 4 } };
```


في هذا المثال تم تخزين العدد ١ في المصفوفة b في الموقع الموجود في تقاطع الصف الأول والعمود الأول، والعدد ٢ في تقاطع الصف الأول والعمود الثاني. وبمعنى آخر تحتوي هذه المصفوفة على صفين، كل صف يحتوي على عنصرين. والشكل يبين محتويات المصفوفة b بعد تنفيذ الجملة السابقة:



المثال التالي: مصفوفة ذات بعدين تحتوي على قيم ، الصفوف تمثل طلبة والاعمدة تمثل درجات الطلبة :

المهام:

1- طباعة المصفوفة مع العناوين الخاصة بها.

2- ايجاد معدل كل طالب موجود في الصف.

```

package example;
import javax.swing.JOptionPane;
import java.util.Scanner;
import javax.swing.*;
public class general
{
    public static void main(String[] args)
    {
        int grades[][] = { { 77, 68, 86, 73 }, { 96, 87, 89, 81 }, { 70, 90, 86, 81 } };
        int sum;
        System.out.println("The array is:");
        System.out.println("\t\t\t[0]\t\t[1]\t\t[2]\t\t[3]");
        for(int i=0; i<grades.length; i++)
        {
            System.out.print("grades["+i+"]"+"\\t");
            for(int j=0; j<grades[i].length; j++)
                System.out.print(grades[i][j]+"\\t");
            System.out.println();
        }
        System.out.println();
        for(int i=0; i<grades.length; i++)
        {
            sum=0;
            for(int j=0; j<grades[i].length; j++)
                sum+=grades[i][j];
            System.out.println("Average for student "+i+" is "+(double)sum/grades[i].length);
        }
    } //end main
} //end class array3

```

Activate Win
Go to Settings to

Output

```

The array is:

grades[0]      77      68      86      73
grades[1]      96      87      89      81
grades[2]      70      90      86      81

Average for student 0 is 76.0
Average for student 1 is 88.25
Average for student 2 is 81.75
BUILD SUCCESSFUL (total time: 0 seconds)

```

تغذية بيانات المصفوفة بشكل يدوي

```
int[][] a = { {1, 2, 3}, {4, 5, 6, 9}, {7}, };
```

	Column 1	Column 2	Column 3	Column 4
Row 1	1 a[0][0]	2 a[0][1]	3 a[0][2]	
Row 2	4 a[1][0]	5 a[1][1]	6 a[1][2]	9 a[1][3]
Row 3	7 a[2][0]			

```
class MultidimensionalArray {
    public static void main(String[] args) {

        int[][] a = {
            {1, 2, 3},
            {4, 5, 6, 9},
            {7},
        };

        System.out.println("Length of row 1: " + a[0].length);
        System.out.println("Length of row 2: " + a[1].length);
        System.out.println("Length of row 3: " + a[2].length);
    }
}
```

```
Length of row 1: 3
Length of row 2: 4
Length of row 3: 1
```

Example: Print all elements of 2d array Using Loop طباعة كل عناصر المصفوفة باستخدام اللوب

```
class MultidimensionalArray {
    public static void main(String[] args) {

        int[][] a = {
            {1, -2, 3},
            {-4, -5, 6, 9},
            {7},
        };

        for (int i = 0; i < a.length; ++i) {
            for(int j = 0; j < a[i].length; ++j) {
                System.out.println(a[i][j]);
            }
        }
    }
}
```

الكود التالي يقوم بحساب قيم الارقام في diagonal :

```
sum=0;
```

```
for(int i=0; i<grades.length; i++)
```

```
for(int j=0; j<grades[i].length; j++)
```

```
if(i==j) sum+=grades[i][j];
```

```
System.out.println("\nThe sume of elements on the main diagonal is: "+sum);
```

```
The array is:
      [0]      [1]      [2]      [3]
grades[0]    77      68      86      73
grades[1]    96      87      89      81
grades[2]    70      90      86      81

Average for student 0 is 76.0
Average for student 1 is 88.25
Average for student 2 is 81.75

The sume of elements on the main diagonal is: 250
BUILD SUCCESSFUL (total time: 0 seconds)
```



Faculty of Computer Science and Mathematics, University of Kufa, Najaf, IRAQ.